06/11/2026

SQAT 6322

SUMMATIVE PROJECT

Mutonda Muofhe
ST10450819

Table of Contents

Smart Public Transport Management System (SPTMS) Quality Assurance Report 2

Introduction 2

1. Implementing Tests 2

1.1 a) Types of tests to be implemented for the SPTMS 2

1.1 b) Challenges and considerations in implementing tests for the SPTMS 3

2. Reporting on Tests 5

2.1 a) Significance of test reporting in the SDLC 5

2.1 b) Sample Test Report for the SPTMS 6

3. Building a Test Environment 9

3.1 a) Steps to build a suitable test environment for the SPTMS and the online payment system 9

3.2 b) Additional considerations and challenges for the SPTMS compared to a single-application product 10

4. Evaluating Security Requirements 11

4.1 a) Three critical security requirements for the integrated SPTMS and online payment system 11

4.2 b) Strategy for evaluating the security requirements 12

Conclusion 13

Reference List 14

Smart Public Transport Management System (SPTMS) Quality Assurance Report

Introduction

Revamp Dreams Solutions has been contracted by the City of Johannesburg to implement a Smart Public Transport Management System (SPTMS). The system consists of three integrated components: a web-based management dashboard used by transport officials to monitor routes, vehicles and schedules; a mobile application that allows commuters to view real-time bus and taxi locations, purchase digital tickets and receive service alerts; and a back-end system integrating GPS tracking, payment processing and user data management.

Because the system must handle high user traffic, manage sensitive personal and financial data, and operate reliably in real time, a structured quality assurance and testing effort is essential. This report describes the tests that should be implemented, the challenges of implementing them, a sample test report, the steps for building a suitable test environment, the evaluation of the system's security requirements, and a conclusion summarising the QA approach (Hooda and Chhillar, 2015).

1. Implementing Tests

1.1 a) Types of tests to be implemented for the SPTMS

At least the following test types should be implemented, each addressing a different quality attribute of the SPTMS:

1. Functional Testing. Functional testing verifies that each component of the SPTMS behaves according to its specified requirements. For example, it checks that a commuter can create an account, view real-time vehicle locations, purchase a digital ticket and receive a service alert, and that a transport official can add a route or edit a schedule on the dashboard. Its purpose is to confirm that the system does what it is supposed to do every user-facing function (ticket purchase, route monitoring, alert generation) produces the correct output for a given input. Techniques include requirement-based test case design, equivalence partitioning and boundary value analysis, executed through unit, integration and system testing (Shi, 2010).

2. Performance and Load Testing. The SPTMS must handle high user traffic, particularly at peak commuting hours when thousands of commuters simultaneously query vehicle locations and purchase tickets, and the GPS integration must update in near real time. Performance testing measures the system's response time, throughput and resource usage under normal load, while load and stress testing simulate thousands of concurrent mobile users and dashboard sessions to verify the system remains responsive and stable, to identify its breaking point, and to confirm the real-time GPS data feed does not degrade. Its purpose is to ensure reliability and acceptable user experience under expected and peak demand.

3. Security Testing. The system manages sensitive user data (personal details, location data, payment information) and processes financial transactions. Security testing attempts to uncover vulnerabilities such as SQL injection, broken authentication, insecure APIs between the mobile app, dashboard and back-end, and weaknesses in the payment integration. It includes penetration testing, authentication/authorisation testing (e.g. verifying that a commuter cannot access another user's tickets or that only authorised officials can change schedules) and data-privacy checks to ensure compliance with POPIA. Its purpose is to protect the integrity and confidentiality of user transactions and prevent financial loss or data breaches (Takanen, Demott, Miller and Kettunen, 2018).

4. Usability Testing. The mobile app must be usable by a wide range of commuters, including users with limited technical experience and users with disabilities (accessibility). Usability testing observes real users performing key tasks finding a bus, buying a ticket, reading an alert and measures task completion rates, time on task and satisfaction, ensuring the system is accessible and delivers a safe, easy user experience.

5. Compatibility/Interoperability Testing. The SPTMS integrates multiple components (web dashboard, mobile app, back-end, GPS devices, payment gateways). Compatibility and integration testing verifies that these components work together correctly across different mobile devices, operating systems, browsers, GPS hardware versions and payment providers, and that data (ticket purchases, GPS coordinates) flows accurately between them.

1.1 b) Challenges and considerations in implementing tests for the SPTMS

Implementing tests for the SPTMS is complex because it is a large, distributed, integrated system rather than a single application:

- **Complexity of integration.** The dashboard, mobile app, back-end, GPS tracking and payment gateway are separate systems communicating in real time. Defects may lie at the interfaces, so integration testing requires all components (or reliable

simulators) to be available simultaneously, and isolating the cause of a failure across multiple vendors' systems is difficult.

- **Real-time behaviour.** GPS location data is continuously changing. Tests that depend on live vehicle positions are non-deterministic the same test may give different results each run so testing must either use simulated GPS data feeds or be designed to validate tolerances rather than exact values.
- **High-load simulation.** Reproducing peak-hour traffic with thousands of concurrent commuters requires expensive load-testing infrastructure, realistic user models and test data, and load tests can be risky and time-consuming to run.
- **Sensitive test data.** Payment and personal data cannot be freely used for testing. Test teams must use masked, anonymised or synthetic data and sandboxed payment accounts, while still keeping the data realistic enough to find defects. POPIA compliance constrains how test data is created and stored.
- **Multiple platforms and devices.** Commuters use many different phone models, OS versions, screen sizes and network conditions (3G/4G/5G, poor coverage areas in Johannesburg). Achieving adequate coverage across combinations is difficult; risk-based prioritisation of device/OS combinations is needed.
- **Availability of a representative test environment.** A production-like environment with real GPS hardware, payment gateway sandboxes, and city infrastructure links is costly and hard to build (discussed in Section 4).
- **Safety and operational criticality.** Incorrect schedule or vehicle data could mislead commuters and officials. Failures affect public services, so testing must emphasise reliability, failover and recovery scenarios, and defects found late carry high operational risk (Ashiq,Masood, Fakhar, and Ahmad,2024).

Influence on the testing approach: these challenges push the approach towards layered testing automated unit and API tests run early and continuously (cheapest to run repeatedly), integration tests supported by simulators/stubs for GPS and payment interfaces, synthetic test data generation for privacy compliance, risk-based prioritisation of test scenarios and device matrices, and a dedicated, production-like test environment with automated load testing. Non-determinism is handled by designing tests around state models and tolerances rather than fixed data, and by recording and replaying GPS streams.

2. Reporting on Tests

2.1 a) Significance of test reporting in the SDLC

Test reporting is the mechanism by which testing activity is made visible and actionable. Its significance in the software development lifecycle includes:

- **Decision support:** objective evidence (pass rates, defect counts, coverage) informs decisions on whether the SPTMS is ready for release, needs further testing, or must launch with known limitations.
- **Progress tracking and visibility:** stakeholders developers, project managers, the City of Johannesburg as client can see what has been tested, what remains, and how quality is trending across builds.
- **Defect communication:** reports highlight where defects are concentrated (e.g. the payment integration), enabling root-cause analysis and targeted rework.
- **Accountability and auditability:** a public-sector system requires documented evidence that functional, security, performance and regulatory requirements were verified essential for compliance and post-release accountability.
- **Process improvement:** metrics such as defect density, defect leakage and test execution trends reveal weaknesses in the development and test process itself.
- **Risk management:** untested areas and open high-severity defects are explicit risk signals for go/no-go decisions (Tuteja and Dubey, 2012).

Key components of a good test report:

1. **Summary/overview** — system under test, version/build, environment, test period, and an overall quality assessment.
2. **Test scope and coverage** — what was tested (requirements/functional areas) and what was not, with test coverage percentages.
3. **Test metrics** — number of test cases planned, executed, passed, failed, blocked; pass rate; execution progress against plan.
4. **Defect information** — number of defects found, grouped by severity and status; details of critical/open defects and their impact.
5. **Risk and issues** — residual risks, untested areas, known limitations, environment or test-data problems.

6. **Recommendations and conclusions** — release readiness verdict, recommended actions, and next steps (e.g. regression testing of fixed defects).

2.1 b) Sample Test Report for the SPTMS

TEST REPORT – Smart Public Transport Management System

Item	Details
System under test	SPTMS – Dashboard, Mobile App, Back-end (Build 3.2)
Test period	12–20 October 2026
Environment	Pre-production test environment (STG-01)
Prepared by	SQA Engineer, Revamp Dreams Solutions
Test types executed	Functional, Integration, Performance/Load, Security, Usability, Compatibility

Summary: 214 test cases executed; 186 passed (87%), 21 failed, 7 blocked (pending GPS simulator fix). 43 defects raised: 5 critical, 11 high, 19 medium, 8 low. Overall assessment: **Not ready for release** until the 5 critical defects are fixed and re-tested.

Test results per test type:

Test type	Cases	Passed	Failed	Key defects
Functional	96	89	7	CR-02, PR-04
Integration	42	35	7	PY-01, PY-02 (critical)
Performance/Load	24	21	3	PR-05, PR-06
Security	28	24	4	SEC-01 (critical), SEC-04
Usability	14	12	2	UX-02
Compatibility	10	5	5	CM-01 to CM-05

Representative test cases, results and issues:

ID	Test case description	Expected result	Actual result	Status	Defect
FUNC-11	Commuter purchases a digital ticket with valid card	Ticket issued, confirmation shown, receipt emailed	Ticket issued; no confirmation shown for Visa cards	Fail	CR-02 (Critical): payment confirmation callback intermittently dropped
FUNC-23	Official adds a new route on the dashboard	Route saved and visible on mobile app within 30 s	Route saved but never appears on the mobile app	Fail	CR-01 (Critical): cache on mobile API not invalidated
INT-07	GPS position update flows from device to back-end to mobile app	Vehicle position updates ≤ 5 s	Position updates only when app is restarted	Fail	PY-02 (Critical): WebSocket push service drops idle connections
LOAD-03	Simulate 10,000 concurrent commuters at peak hour	Response time ≤ 3 s, error rate < 1%	Response time 9.4 s, error rate 4.2% at 6,500 users	Fail	PR-05 (High): ticket-purchase API bottleneck — DB connection pool exhausted
SEC-02	Attempt SQL injection on login field	Input rejected, request logged	Input reflected in error message; server 500	Fail	SEC-01 (Critical): unvalidated input on login API
SEC-09	Commuter attempts to view another user's ticket	Access denied	Access denied, attempt logged	Pass	—

PERF-08	Back-end CPU/memory at sustained 5,000 users	Usage stable, no degradation	Stable within thresholds	Pass	—
UX-01	First-time commuter finds and buys a ticket in ≤ 3 minutes	90% task success	93% success, avg 2.1 min	Pass	—
UX-05	Low-literacy user reads a service alert	Alert understood by ≥ 80% of participants	Only 55% understood "route diversion" wording	Fail	UX-02 (Medium): alert wording too technical
CM-03	Ticket purchase on Android 9 low-end device	Completes in ≤ 10 s	App crashes on payment screen	Fail	CM-01 (High): crash on older Android WebView versions

Issues, defects and conclusions: The five critical defects (CR-01, CR-02, PY-02, SEC-01 and PR-05) all affect ticket purchasing or real-time data delivery the two core capabilities of the SPTMS and must be fixed and regression tested before release. Load test results show the system does not yet meet the peak-hour requirement, and the SQL injection vulnerability must be remediated and retested with a follow-up penetration test.

Recommendation: release is blocked pending critical fixes, targeted performance remediation and security retesting.

3. Building a Test Environment

3.1 a) Steps to build a suitable test environment for the SPTMS and the online payment system

1. **Analyse requirements.** Review the SPTMS requirements, architecture and non-functional requirements (expected 10,000+ peak users, ≤ 3 s response, real-time GPS updates) to determine what the test environment must support.
2. **Define the environment specification.** Document the required components: web server for the dashboard, application servers, database server, mobile app build/test devices, GPS device simulators, payment gateway sandbox, and monitoring tools. Define hardware, software and network configuration standards.
3. **Acquire and configure hardware.** Provision servers (physical or cloud VMs/containers) sized to represent production proportionally application servers, database server with production-equivalent storage, and real or emulated mobile devices for app testing.
4. **Install and configure software.** Deploy the dashboard, back-end services, database, message queues, GPS ingestion services and mobile app builds (test builds for Android/iOS). Version-control configuration so environments can be rebuilt identically.
5. **Configure the network.** Replicate production network conditions: bandwidth limits, latency (e.g. simulate 3G/4G for mobile users), firewalls, load balancers, and secure zones separating payment processing from other traffic.
6. **Integrate third-party services in sandbox mode.** Connect the payment gateway in test/sandbox mode (no real money), and configure GPS device simulators that replay recorded or generated vehicle tracks so real-time behaviour can be tested deterministically.
7. **Prepare test data.** Generate synthetic commuters, tickets, routes, schedules and vehicle data; mask or anonymise any production-derived data to comply with POPIA. Include boundary and invalid data sets.
8. **Install test and monitoring tools.** Install test management (e.g. Jira/Xray), automation (Selenium, Appium, JMeter/LoadRunner), defect tracking, log aggregation and application performance monitoring so every test run is observable.

9. **Validate the environment.** Run smoke tests to confirm all components integrate correctly (app → back-end → GPS simulator → payment sandbox) before formal testing begins.
10. **Document and maintain.** Record configurations, credentials (secured), refresh/restore procedures, and set up environment refresh between test cycles so results are reproducible (Behera,Sahoo and Jena, 2021).

3.2 b) Additional considerations and challenges for the SPTMS compared to a single-application product

Unlike testing a single desktop or standalone application, the SPTMS is a multi-component, distributed, public-facing system, which introduces additional considerations:

- **Multi-system integration:** the environment must host three interdependent components (dashboard, mobile app, back-end) plus external dependencies (GPS hardware, payment gateway, city systems). All interfaces must be replicated or simulated; a single application needs only one runtime.
- **Real-time and location dependencies:** simulating GPS vehicle movements requires device simulators or recorded GPS replay streams, and the test network must reproduce real-time message queues and push services none of which exists for a single app.
- **Mobile device diversity:** a representative device farm (multiple Android/iOS versions, low-end handsets used by typical commuters) is required, versus one or two platforms for a single app.
- **Load and network realism:** the environment must support generating thousands of concurrent users and simulating Johannesburg's variable mobile network conditions; a single app is usually tested by one user at a time.
- **Payment integration security:** the payment subsystem requires a certified sandbox with PCI-DSS-compliant handling of test card data, isolated from other test components.
- **Scale and cost:** a production-like environment with servers, simulators, devices, licences and cloud load-generation capacity is significantly more expensive and slower to build than a single-application test bed.

- **Data privacy at city scale:** large volumes of realistic commuter data must be synthetic or anonymised (POPIA), and access to the environment must be strictly controlled.
- **Operational criticality:** because the system serves the public, the environment must also support failover, disaster-recovery and negative testing without risk to any live service requiring close coordination with the City of Johannesburg.

4. Evaluating Security Requirements

4.1 a) Three critical security requirements for the integrated SPTMS and online payment system

1. Secure authentication and authorisation. Every user commuter, transport official, administrator must prove their identity with strong authentication (passwords with MFA, secure session tokens), and the system must enforce role-based access control so users can only perform actions their role permits (a commuter cannot alter schedules; only authorised officials can modify routes). This is essential because weak authentication would let attackers take over commuter accounts and drain stored payment methods, or let unauthorised users tamper with public transport operations directly compromising the security and integrity of user transactions.

2. Encrypted transmission and storage of sensitive data. All data must be encrypted in transit (TLS 1.2+) between the mobile app, dashboard, back-end, GPS devices and payment gateway, and sensitive fields (card data, personal information, location history) must be encrypted at rest with managed keys. This protects commuter personal and financial data from interception (e.g. over mobile networks or public Wi-Fi) and ensures that even if a database is compromised, transaction data remains confidential and unaltered preserving transaction integrity and POPIA compliance (Hayrapetian, and Raje, 2018).

3. Payment integrity and secure transaction processing. The online payment system must guarantee that every transaction is processed exactly once, cannot be tampered with in transit, and complies with PCI-DSS: card data must be tokenised and handled by the certified payment gateway (never stored raw by the SPTMS), transactions must be atomic and auditable (immutable logs), and replay/duplicate-payment attacks must be prevented. This is essential because the system handles commuters' money: any weakness could

enable fraudulent ticket purchases, double-charging, or manipulation of fare records, destroying user trust and exposing the City to financial and legal liability.

(Supporting requirements: audit logging and monitoring of all security events; vulnerability management and patching; API security the back-end APIs between the app and payment system are prime attack surfaces and must validate all input, use signed requests and rate limiting.)

4.2 b) Strategy for evaluating the security requirements

A layered security testing strategy should be applied throughout the lifecycle:

1. **Security requirements review and threat modelling.** Before testing, review the requirements above and build a threat model (e.g. STRIDE) of the SPTMS architecture, identifying attack surfaces: mobile app APIs, dashboard login, payment gateway integration, GPS data ingestion. This determines what to test and prioritises high-risk areas.
2. **Static analysis (SAST) and secure code review.** Run automated static analysis on the app, dashboard and back-end code to detect injection flaws, hard-coded credentials, insecure crypto use, and combine it with manual review of authentication, session management and payment code paths.
3. **Dynamic analysis (DAST) and penetration testing.** Against the test environment, run dynamic scans and conduct a manual penetration test simulating real attackers: SQL injection on login and ticket APIs, broken authentication/authorisation tests (attempting to access another commuter's tickets or an official's dashboard functions), session hijacking, and manipulation of payment messages.
4. **Authentication and authorisation testing.** Verify MFA enforcement, password policies, token expiry, and systematically test role boundaries e.g. replay an official's API token from the commuter app, attempt horizontal privilege escalation between commuters, and confirm access is denied and logged.
5. **Encryption and data-protection verification.** Inspect all traffic to confirm TLS is used and correctly configured (certificate validity, no downgrade), verify no card data is stored by the SPTMS (PCI-DSS scope check), verify data at rest is encrypted, and inspect logs/memories/dumps for leakage of sensitive data.
6. **Payment-specific testing.** Use the payment gateway sandbox to test transaction integrity: duplicate/replayed transactions rejected, interrupted transactions handled

atomically (no ticket issued without payment and vice versa), refund and reconciliation accuracy, and input validation on all payment fields.

7. **Security regression and monitoring validation.** Automate the critical security tests into the regression suite so they run on every build, and validate that audit logging and alerting correctly capture the attack attempts made during testing.
8. **Acceptance criteria:** results are reported against the three requirements any failed check is a blocking defect; retest after remediation and obtain sign-off before release (Fabian, Gürses, and Schmidt, 2010).

Conclusion

The SPTMS is a large, integrated, real-time public transport platform on which the City of Johannesburg and thousands of commuters will depend. This report has shown that assuring its quality requires a comprehensive, layered testing strategy: functional, integration, performance, security, usability, compatibility and regression testing, each targeting a specific quality attribute of the dashboard, mobile app and back-end. Implementing these tests is challenging because of the system's distributed architecture, real-time GPS and payment integrations, sensitive data obligations under POPIA, and the need to simulate peak-hour loads challenges that shape a risk-based, automated and simulator-driven approach. A well-built test environment replicating hardware, software, network conditions and third-party services in sandbox mode is essential to produce reliable, reproducible results, and thorough test reporting converts those results into evidence-based release decisions. Finally, the security of user transactions secured through strong authentication, encryption, and PCI-DSS-compliant payment processing, validated by threat modelling, static/dynamic analysis and penetration testing must be treated as a non-negotiable gate for release. With these QA practices in place, Revamp Dreams Solutions can deliver an SPTMS that is functional, secure, performant and trustworthy for the City of Johannesburg and its commuters.

Reference List:

Ashiq, S., Masood, A.B., Fakhar, M.H., Iqbal, M.W., Nazir, Z., Muhammad, H.A.B., Rahman, S.U. and Ahmad, S.Z., 2024. Challenges and barriers to software testing. *Bulletin of Business and Economics (BBE)*, 13(1).

Behera, R.K., Sahoo, A.K. and Jena, A., 2021. A resourceful approach in security testing to protect electronic payment system against unforeseen attack. In: *Research Anthology on Artificial Intelligence Applications in Security* (pp. 1279–1302). IGI Global Scientific Publishing.

Fabian, B., Gürses, S., Heisel, M., Santen, T. and Schmidt, H., 2010. A comparison of security requirements engineering methods. *Requirements Engineering*, 15(1), pp. 7–40.

Hayrapetian, A. and Raje, R., 2018, February. Empirically analyzing and evaluating security features in software requirements. In: *Proceedings of the 11th Innovations in Software Engineering Conference* (pp. 1–11).

Hooda, I. and Chhillar, R.S., 2015. Software test process, testing types and techniques. *International Journal of Computer Applications*, 111(13), pp. 10–14.

International Software Testing Qualifications Board (ISTQB), 2023. *Certified Tester Foundation Level Syllabus v4.0*. Brussels: ISTQB.

ISO/IEC/IEEE, 2022. *ISO/IEC/IEEE 29119-1:2022 Software and systems engineering — Software testing — Part 1: General concepts*. Geneva: International Organization for Standardization.

Kaner, C., Bach, J. and Pettichord, B., 2013. *Lessons Learned in Software Testing: A Context-Driven Approach*. Hoboken, NJ: John Wiley & Sons.

Kaner, C., Falk, J. and Nguyen, H.Q., 2011. *Testing Computer Software*. 3rd ed. Hoboken, NJ: John Wiley & Sons.

Myers, G.J., Sandler, C. and Badgett, T., 2011. *The Art of Software Testing*. 3rd ed. Hoboken, NJ: John Wiley & Sons.

Naik, K. and Tripathy, P., 2018. *Software Testing and Quality Assurance: Theory and Practice*. 2nd ed. Hoboken, NJ: John Wiley & Sons.

National Institute of Standards and Technology (NIST), 2020. *Security and Privacy Controls for Information Systems and Organizations (NIST Special Publication 800-53 Rev. 5)*. Gaithersburg, MD: NIST.

OWASP Foundation, 2023. *OWASP Web Security Testing Guide v4.2*. [Online] Available at: https://owasp.org/www-project-web-security-testing-guide/ [Accessed 17 October 2026].

Payment Card Industry Security Standards Council (PCI SSC), 2022. *Payment Card Industry Data Security Standard (PCI DSS) v4.0*. Wakefield, MA: PCI Security Standards Council.

Republic of South Africa, 2013. *Protection of Personal Information Act 4 of 2013*. Pretoria: Government Gazette.

Shi, M., 2010. Software functional testing from the perspective of business practice. *Computer and Information Science*, 3(4), p. 49.

Takanen, A., Demott, J.D., Miller, C. and Kettunen, A., 2018. *Fuzzing for Software Security Testing and Quality Assurance*. Artech House.

Tuteja, M. and Dubey, G., 2012. A research study on importance of testing and quality assurance in software development life cycle (SDLC) models. *International Journal of Soft Computing and Engineering (IJSCE)*, 2(3), pp. 251–257.